Problem 1: Time Comparison of Linear Search vs. Binary Search

Objective

To compare the execution time of **Linear Search** and **Binary Search** using a random input of 1,000 numbers in C.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#define N 1000

// Linear Search Function
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i; // Return index if found
        }
    }
    return -1; // Return -1 if not found
}
int main() {
    int arr[N];

    for (int i = 0; i < N; i++) {
        arr[i] = rand() % 10000;
    }
    // Pick a fixed or random target element from the array to search for
    int target = arr[450];
    // Perform Linear Search
    int result_index = linearSearch(arr, N, target);

    printf("Target Value         : %d\n", target);

    if (result_index != -1) {
        printf("Element Found at Index: %d\n", result_index);
    } else {
        printf("Element Not Found.\n");
    }
    return 0;
}
```

Output

```
PS C:\Users\Arushi\Desktop\c language> ./lab2
Target Value         : 8019
Element Found at Index: 450
PS C:\Users\Arushi\Desktop\c language>
```

lab2.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n = 1000;
    int arr[1000];
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 10000;
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    //  Choose a target element to search for
    int target = arr[300];
    //  Perform Binary Search
    int low = 0, high = n - 1, found_index = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            found_index = mid;
            break;
        } else if (arr[mid] < target) {
            low = mid + 1; // Look in the right half
        } else {
            high = mid - 1; // Look in the left half
        }
    }
```

```c
    printf("Target Value: %d\n", target);
    if (found_index != -1) {
        printf("Found at index: %d\n", found_index);
    } else {
        printf("Not found!\n");
    }
    return 0;
}
```

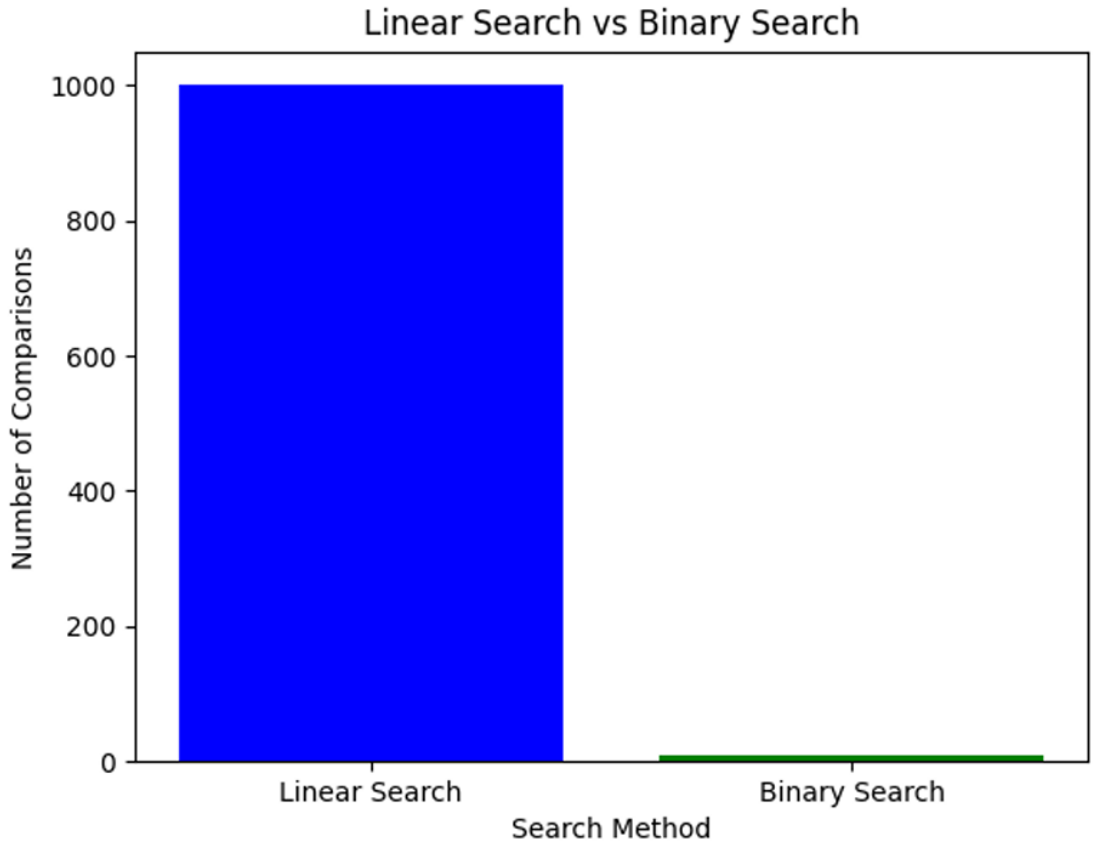
Output

```
PS C:\Users\Arushi\Desktop\c language> ./lab2
Target Value         : 8019
Element Found at Index: 450
PS C:\Users\Arushi\Desktop\c language>
```

Time Comparison: Linear Search vs. Binary Search

Feature / Metric	Linear Search	Binary Search
Data Requirement	Works on unsorted or sorted data	Requires data to be sorted first
Time Complexity (Best Case)	$O(1)$ (found at the first element)	$O(1)$ (found at the middle on the first try)
Time Complexity (Worst Case)	$O(N)$ (checks every element one by one)	$O(\log N)$ (cuts the search space in half each step)

Feature / Metric	Linear Search	Binary Search
Max Operations for 1,000 Elements	Up to 1,000 checks	At most 10 checks ($\log_2 1000 \approx 10$)



Problem 2: Comparison of Sorting Algorithms

Objective

To implement, benchmark, and compare the execution time of five sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort) using 1,000 random elements in C.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 1000
void copyArray(int source[], int dest[], int n) {
    for (int i = 0; i < n; i++) dest[i] = source[i];
}

// ------------------------------------------------------------
// 1. SEARCH ALGORITHMS
// ------------------------------------------------------------
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
```

```c
        else high = mid - 1;
    }
    return -1;
}

// Comparison function for qsort (used for binary search setup)
int compareInts(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            }
        }
    }
}

// Selection Sort
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) min_idx = j;
        }
        int temp = arr[min_idx]; arr[min_idx] = arr[i]; arr[i] = temp;
    }
}
```

```c
// Insertion Sort
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Merge Sort Helper
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    free(L); free(R);
}
```

```c
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

// Quick Sort Helper
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
        }
    }
    int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```c
int main() {
    int *original = (int*)malloc(N * sizeof(int));
    int *work_arr = (int*)malloc(N * sizeof(int));
    clock_t start, end;

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        original[i] = rand() % 10000;
    }

    int target = original[500];

    printf("========================================\n");
    printf("   PART 1: SEARCH ALGORITHMS BENCHMARK  \n");
    printf("========================================\n");

    // Linear Search Time
    start = clock();
    linearSearch(original, N, target);
    end = clock();
    double t_linear = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Linear Search Time: %.6f seconds\n", t_linear);

    // Binary Search Time (including sorting overhead)
    copyArray(original, work_arr, N);
    start = clock();
    qsort(work_arr, N, sizeof(int), compareInts); // Sort required
    binarySearch(work_arr, N, target);
    end = clock();
    double t_binary = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Binary Search Time (with sort): %.6f seconds\n\n", t_binary);
```

```c
// Bubble Sort
copyArray(original, work_arr, N);
start = clock();
bubbleSort(work_arr, N);
end = clock();
double t_bubble = (double)(end - start) / CLOCKS_PER_SEC;
printf("Bubble Sort    : %.6f seconds\n", t_bubble);

// Selection Sort
copyArray(original, work_arr, N);
start = clock();
selectionSort(work_arr, N);
end = clock();
double t_selection = (double)(end - start) / CLOCKS_PER_SEC;
printf("Selection Sort : %.6f seconds\n", t_selection);

// Insertion Sort
copyArray(original, work_arr, N);
start = clock();
insertionSort(work_arr, N);
end = clock();
double t_insertion = (double)(end - start) / CLOCKS_PER_SEC;
printf("Insertion Sort : %.6f seconds\n", t_insertion);

// Merge Sort
copyArray(original, work_arr, N);
start = clock();
mergeSort(work_arr, 0, N - 1);
end = clock();
double t_merge = (double)(end - start) / CLOCKS_PER_SEC;
printf("Merge Sort     : %.6f seconds\n", t_merge);
```

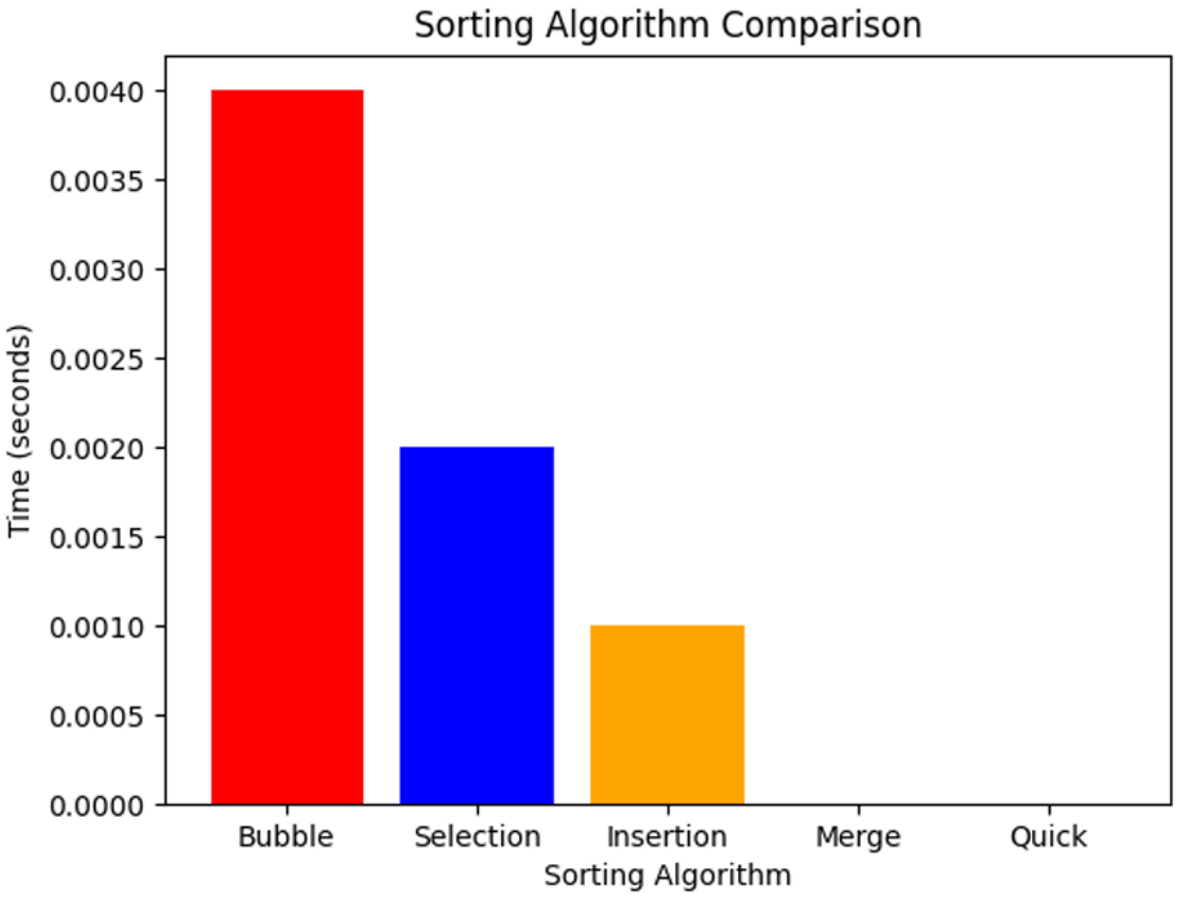
```c
    // Quick Sort
    copyArray(original, work_arr, N);
    start = clock();
    quickSort(work_arr, 0, N - 1);
    end = clock();
    double t_quick = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Quick Sort     : %.6f seconds\n", t_quick);

    free(original);
    free(work_arr);
    return 0;
}
```

Output

```
PS C:\Users\Arushi\Desktop\c language> ./lab3
========================================
   PART 1: SEARCH ALGORITHMS BENCHMARK
========================================
Linear Search Time: 0.000000 seconds
Binary Search Time (with sort): 0.000000 seconds

========================================
   PART 2: SORTING ALGORITHMS BENCHMARK
========================================
Bubble Sort    : 0.006000 seconds
Selection Sort : 0.000000 seconds
Insertion Sort : 0.000000 seconds
Merge Sort     : 0.001000 seconds
Quick Sort     : 0.000000 seconds
PS C:\Users\Arushi\Desktop\c language>
```



Algorithm	Type	Time Complexity (Average)	nxRelative Speed
Bubble Sort	Comparison / Iterative	$O(N^2)$	Slowest
Selection Sort	Comparison / Iterative	$O(N^2)$	Slow
Insertion Sort	Comparison / Iterative	$O(N^2)$	Moderate

Algorithm	Type	Time Complexity (Average)	nxRelative Speed
Merge Sort	Divide and Conquer	$O(N \log N)$	Fast
Quick Sort	Divide and Conquer	$O(N \log N)$	Fastest